

Adaptive Software-Defined Sonar Transmitter

Complete Technical Documentation

Smart India Hackathon 2026

Problem Statement ID: 26058

Problem Statement Title: Low-Power, Real-Time Adaptive Software-Defined Sonar Transmitter Payload for AUV

PS Category: Hardware

Team Name: CODE RED

Code and design files: github.com/Agent-Bid/SIH_2026

October 2026

Contents

1. Problem and Motivation
2. Scientific Basis
3. System Architecture
4. Hardware: Specifications and Rationale
5. Decision Engine (ESP32-S3 Firmware)
6. Machine Learning Model
7. ESP32 to FPGA Link
8. FPGA Waveform Engine
9. DAC and Analog Output
10. Verification Strategy
11. Results
12. Bill of Materials and Cost
13. Bench Testing Performed
14. Limitations and Honesty Ledger
15. Roadmap
16. How to Build and Run
17. Repository Layout
18. References

1. Problem and Motivation

An autonomous underwater vehicle (AUV) uses sonar to see, but the water the sound travels through keeps changing under it. The problem statement asks for a transmitter payload that is low power, real time, adaptive and software defined. This section explains why each of those words matters.

1.1 A sonar tuned for one ocean

- The speed of sound in seawater ranges from about 1,450 to 1,550 m/s with temperature, salinity and depth, and an AUV crosses all three on every dive.
- Absorption grows steeply with frequency and with temperature. At the surface in 35 ppt water, 300 kHz loses about 66 dB/km at 5 °C but 137 dB/km at 30 °C, and 500 kHz loses 129 to 199 dB/km over the same temperatures (Francois-Garrison, section 2.2).
- Suspended sediment adds its own loss, and it changes from one plume to the next.
- A conventional sonar is set once on the bench: one frequency, one pulse length, one power. In clear, cold water it wastes power. In warm or murky water it may not reach the target at all.

1.2 The resolution versus range dilemma

- Range resolution is $c / (2B)$, where B is the bandwidth of the pulse. A 120 kHz sweep resolves about 0.6 cm; a 40 kHz sweep about 1.9 cm.
- High frequencies give fine detail but die quickly. Low frequencies reach far but blur.
- The right trade-off changes ping by ping with the water and the distance to the target, so no single setting is right for a whole mission.

1.3 Battery endurance

- An AUV's mission time is set by its battery. Every joule the sonar spends beyond what the link needs is mission time lost.
- In our simulated mission, a fixed sonar sized for the worst case (120 kHz, 20 ms pulse, full power) spends **178 times** the energy per ping of the adaptive design. A cheaper fixed setting (300 kHz, 1 ms) saves energy but loses the target on **29 %** of pings (section 11.3).

1.4 Motion

- A moving target shifts every frequency by the Doppler effect. Over one pulse the phase drifts by $f_d \times T$ cycles, where f_d is the Doppler shift and T the pulse length.
- Phase-coded pulses (BPSK) give the sharpest result when nothing moves but fall apart quickly with motion. Frequency sweeps tolerate motion much better. The best waveform therefore depends on how fast the target is moving.

1.5 The problem as a table

Dimension	Fixed sonar today	Gap	This project's response
Frequency and pulse	Chosen once, on the bench	Wrong for most of the water an AUV meets	Chosen for every ping from a physics model of the water: 12 profiles, 100 to 500 kHz, 1 to 20 ms
Power	Sized for the worst case	Wastes battery on easy pings	Just enough power to close the link plus a 3 dB margin, capped by the battery's state of charge
Modulation	One type	Either blurs, or fails with motion	BPSK, LFM or geometric sweep, chosen from the target's motion
Real time	Not applicable	Decisions must keep up with the pings	6 decisions a second on the ESP32-S3; the FPGA plays each one within a ping period
Software defined	Fixed analog chirp generator	Cannot change in the field	Every waveform parameter is a number in a 57-byte packet; the FPGA synthesises it digitally
Trust in automation	Not applicable	A neural network alone is a black box	The neural networks propose; the physics checks and can override every pick

2. Scientific Basis

Every decision the transmitter makes rests on published underwater acoustics models. This section lists them, why each was chosen, and how we checked our implementation.

2.1 Sound speed

- Mackenzie's nine-term equation (1981) gives the speed of sound from temperature, salinity and depth.
- Our implementation is unit-tested against the published check value: 1550.744 m/s at 25 °C, 35 ppt and 1,000 m.

2.2 Absorption

- We use **Francois and Garrison (1982)**, which is published for 200 Hz to 1 MHz and so covers our whole 100 to 500 kHz band. It depends on temperature, salinity, depth and pH, which is exactly what changes as the vehicle moves.
- We deliberately did not use Thorp's formula, which is common in simpler designs. It is a low-frequency fit and ignores temperature, salinity and depth, so it badly underestimates loss in warm water at our frequencies.
- Some public copies of Francois-Garrison carry a digit error in the cold-water pure-water term (4.397e-4 instead of 4.937e-4). We use the correct value and guard it with a test: the formula's two temperature branches only join smoothly at 20 °C with the correct digit.
- Every other term matches an independent implementation (MB-System's mbabsorption) to six digits.

2.3 Turbidity loss

- Suspended particles add attenuation. We model it as 0.15 dB/km per NTU at 100 kHz, growing in proportion to frequency.
- The figure is anchored to Richards, Heathershaw and Thorne (1996), who measured about 3 dB of extra loss over 100 m at 100 kHz for 0.2 kg/m³ of suspended sediment.
- The conversion from NTU to sediment concentration is site specific, so this coefficient needs calibration in real water (section 14).

2.4 Noise

- Above about 100 kHz, the ocean's background noise is dominated by thermal noise, which rises with frequency (Mellen, 1952: $-15 + 20 \log f$ dB, f in kHz).
- Wind-driven noise is added for a 5 m/s wind, and the receiver adds a 6 dB noise figure.

2.5 The sonar equation and pulse compression

For each candidate waveform the firmware computes:

- Transmission loss: $TL = 20 \log R + (\text{absorption} + \text{turbidity loss}) \times R$
- Source level: $SL = 170.8 + 10 \log(\eta \times P) + DI$, for electrical power P , efficiency η and directivity DI
- Output signal-to-noise ratio after pulse compression: $SNR = SL + 10 \log T - TL - N0$

- Required: 12 dB detection threshold plus a 3 dB margin for refraction, multipath and filter loss

The key consequence is that after pulse compression the SNR depends on the **pulse energy** (power $\times$ time), not on the bandwidth. Three design rules follow from that one fact:

- Bandwidth buys range resolution and costs nothing in SNR.
- A longer pulse or more power buys SNR.
- A lower frequency always needs less energy, because the water absorbs less and the thermal noise is lower.

2.6 Waveforms

Waveform	What it is	Strength	Weakness	Used when
CW tone	A single frequency	Simplest	Resolution limited by the pulse length	Test mode
LFM chirp	Frequency rises linearly across the band	Resolution $c / 2B$; tolerates moderate Doppler	Range and Doppler are coupled	Doppler drift between 0.25 and 1 cycle
Geometric sweep	Frequency multiplied by a constant ratio at each step	Most tolerant of motion in our design	Slightly more complex to generate	Doppler drift above 1 cycle
BPSK, Barker-13	A tone whose phase flips 180° in a 13-chip pattern	Sharpest: sidelobes 1/13 of the peak (-22.3 dB)	Breaks down quickly with motion	Target nearly still (drift at most 0.25 cycle)

2.7 Window

Every pulse is shaped by a Hann window: it starts and ends at zero instead of switching on and off abruptly. This keeps the transmitted spectrum inside the chosen band, which matters both for the transducer and for not spilling energy where it is not wanted (Harris, 1978).

2.8 Direct digital synthesis

The FPGA generates waveforms by direct digital synthesis (Tierney, Rader and Gold, 1971). A 32-bit phase accumulator advances by a frequency tuning word every clock, and the top bits of the phase index a sine table. At our 50.142857 MHz clock the frequency resolution is about 0.012 Hz, so any frequency, sweep rate or phase flip in the band can be produced exactly.

2.9 Sigma-delta conversion

Our current digital-to-analog stage is a 1-bit sigma-delta modulator: a single pin switches between 0 and 3.3 V at 50 MHz, and the share of time it spends high follows the waveform. A low-pass filter turns that into the analog signal. The order of the modulator sets how strongly its quantisation noise is pushed to high frequencies, where the filter removes it (Schreier and Temes, 2005). We use a second-order modulator (section 9).

3. System Architecture

The system is a straight chain from water conditions to analog waveform, with two measurement paths alongside it that let us check every stage on the bench.

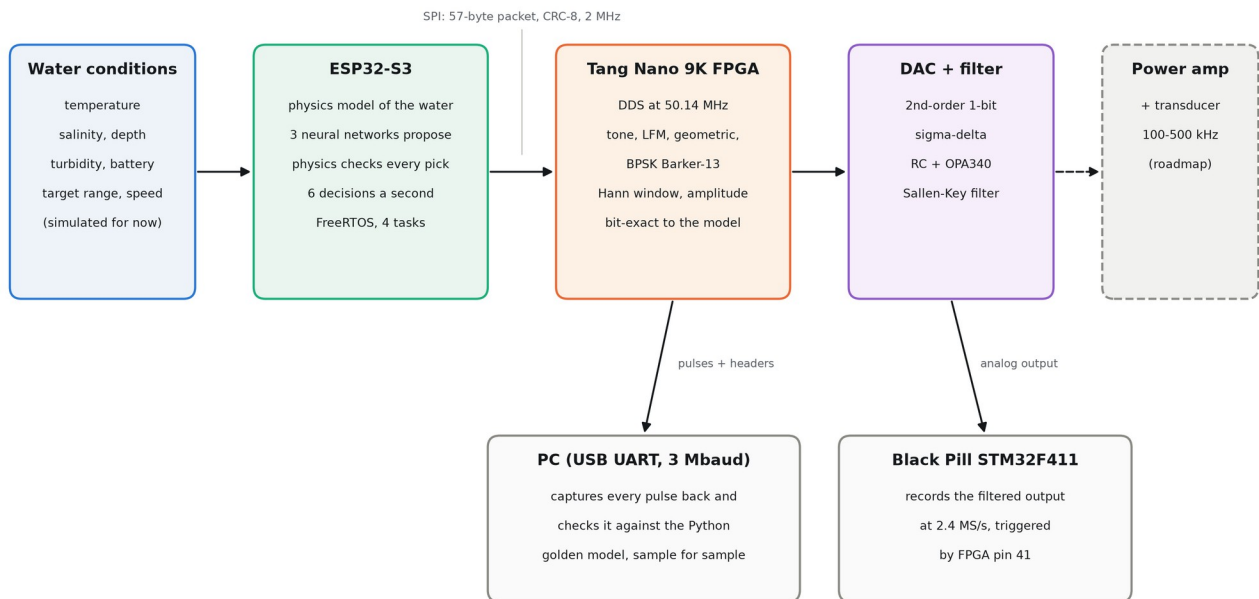


Figure 1. System architecture. Solid boxes are built and verified; the dashed box is the next stage on the roadmap.

Layer	What it owns	Takes input from	Hands off to
1. Inputs	Temperature, salinity, depth, turbidity, battery, target range and speed (simulated for now)	Environment or simulation	Layer 2
2. Decision (ESP32-S3)	Physics model, three neural networks, selection rules, FreeRTOS scheduling	Layer 1	Layer 3
3. Link	57-byte packet over SPI with sync byte, version and CRC-8	Layer 2	Layer 4
4. Waveform engine (FPGA)	Phase accumulators, sweeps, phase code, window, amplitude: one sample per 20 ns	Layer 3	Layer 5
5. Conversion	Second-order sigma-delta DAC, RC stage and OPA340 Sallen-Key filter	Layer 4	Layer 6
6. Transmit (roadmap)	Power amplifier and transducer	Layer 5	Water

- A packet that fails its checks is never played. The FPGA counts it as bad and keeps playing the last good packet.
- A new packet is only applied between pulses (double buffering), so a pulse is never changed halfway through.
- Each layer can be tested on its own. The PC can stand in for the ESP32, the FPGA can send any pulse back to the PC, and the Black Pill can record the analog output.

4. Hardware: Specifications and Rationale

4.1 Decision controller: ESP32-S3

Item	Detail
Processor	Dual-core Xtensa LX7 at up to 240 MHz with a single-precision floating-point unit
Why this part	Runs the physics model and three small neural networks well within a ping period; floating point keeps the C model identical to the Python one; native USB and SPI; low cost
Software	Arduino core on FreeRTOS, four tasks (section 5.8)
Interfaces used	SPI to the FPGA (GPIO 12 MOSI, 13 SCK, 14 CS), native USB for the console, GPIO 1 ADC for the slow-motion DAC view

4.2 Waveform engine: Sipeed Tang Nano 9K (Gowin GW1NR-9 FPGA)

Item	Detail
Resources	8,640 LUT4, 6,480 flip-flops, 26 block RAMs
Clock	27 MHz crystal, PLL to 50.142857 MHz ($27 \times 13 / 7$)
Our use	49 % of LUTs, 50 % of flip-flops, 18 of 26 block RAMs; timing closes at 65.7 MHz against the 50.14 MHz needed
Why an FPGA	Every sample comes out on an exact 20 ns grid and matches a reference model bit for bit. A microcontroller cannot generate 500 kHz sweeps, phase flips and windows with that precision while also doing everything else.
Toolchain	Fully open source: yosys, nextpnr-himbaechel, Apyscula, openFPGALoader

4.3 Digital-to-analog stage

- **In use now:** a second-order 1-bit sigma-delta on FPGA pin 40, followed by an RC stage (1 k Ω , 100 pF) and an OPA340 Sallen-Key low-pass filter (section 9).
- **OPA340:** single supply 2.7 to 5.5 V, rail-to-rail input and output, 5.5 MHz gain-bandwidth, 6 V/ μ s slew rate. It runs from the same 3.3 V as the logic, and its slew rate comfortably covers our largest signal (about 3 V/ μ s at 500 kHz).
- **Why not the DAC0808 board we started with:** it needs a negative supply (-5 to -15 V), and the TL084 op-amps on that board (3 MHz gain-bandwidth) are too slow for a 650 kHz fifth-order Butterworth filter. The board is shelved, not abandoned.
- **For the final transmitter:** the FPGA already drives an 8-bit parallel DAC bus (pins 25 to 34) with a sample clock (pin 35), ready for a single-supply high-speed DAC of the DAC908 or AD9708 class.

4.4 Measurement: Black Pill (STM32F411CE)

- Its 12-bit ADC samples the filter output at **2.4 MS/s**: the chip runs at 72 MHz so that the ADC clock is exactly 36 MHz, with 15 clocks per sample.

- DMA fills a 40,000-sample ring buffer (16.6 ms). A rising edge on FPGA pin 41 marks each pulse's start, so every recording is aligned to its pulse.
- This gives the real 100 to 500 kHz waveform on a laptop, with no slow-motion trick and no oscilloscope needed.

4.5 FPGA pin map

Signal	FPGA pin	Connected to
Clock	52	27 MHz oscillator
SPI MOSI / SCK / CS	37 / 38 / 36	ESP32 GPIO 12 / 13 / 14
UART TX / RX	17 / 18	On-board USB UART to the PC (3 Mbaud)
DAC data [7:0]	25, 26, 27, 28, 29, 30, 33, 34	Parallel DAC (offset binary), for the final transmitter
DAC clock	35	Parallel DAC sample clock (inverted system clock)
Sigma-delta output	40	RC stage, then the Sallen-Key filter
Pulse trigger	41	High for the whole pulse: Black Pill PB0 and the oscilloscope trigger
Buttons	3 / 4	S1 steps through test presets; S2 captures the next pulse
LEDs	10, 11, 13, 14, 15, 16	BPSK, capture armed, recording, good packet, pulse active, geometric

4.6 Power

- Each board runs from its own USB supply, with all grounds joined. Supply rails are never tied together.
- The FPGA is powered first (or together with the ESP32), so the ESP32 never drives signals into an unpowered FPGA.
- The op-amp runs from 3.3 V and draws well under 1 mA.

5. Decision Engine (ESP32-S3 Firmware)

For every ping, six times a second, the ESP32 reads its inputs, works out what the water is doing to sound at each frequency, picks the waveform, sets the power and builds the packet for the FPGA.

5.1 Inputs

Input	Range	Source today
Temperature	2 to 30 °C	Simulated
Salinity	25 to 40 ppt	Simulated
Depth	0 to 1,000 m	Simulated
Turbidity	0 to 100 NTU	Simulated
Battery	0 to 100 %	Simulated
Target range	10 to 2,000 m	Simulated per contact
Relative speed	0 to 10 m/s	Simulated per contact

The inputs can also be typed on the serial console or read from potentiometers, which is useful for testing one condition at a time.

5.2 Simulated water conditions

Until real sensors are fitted, a built-in simulation drives the system through a realistic mission:

- The vehicle dives and climbs between 20 and 350 m every 20 mission-minutes, through a thermocline (22 °C down to 5 °C) and a halocline (34.4 to 35.2 ppt).
- Turbidity is a background level plus drifting sediment plumes and a muddy layer near the bottom.
- The battery drains and recharges at 15 %.
- Three contacts are pinged in turn: A near (15 to 180 m) and lively, B mid-range (250 to 600 m) and nearly still, C far (450 to 1,000 m) and moving. Ranges and speeds follow smooth random motions.
- Every reading carries a little sensor noise, and mission time runs 10 times faster than real time so a demo covers a lot of ocean.
- It is deterministic from its seed. A Python copy reproduces it to within 0.001 over 20,000 pings, so the PC can work out the conditions behind any packet from its sequence number.

5.3 Waveform profiles

Profiles	Band	Centre	Bandwidth	Pulse lengths	Resolution
X-S / X-M / X-L	380 to 500 kHz	440 kHz	120 kHz	1 / 5 / 20 ms	about 0.6 cm
H-S / H-M / H-L	270 to 330 kHz	300 kHz	60 kHz	1 / 5 / 20 ms	about 1.3 cm
M-S / M-M / M-L	180 to 220 kHz	200 kHz	40 kHz	1 / 5 / 20 ms	about 1.9 cm
L-S / L-M / L-L	100 to 140 kHz	120 kHz	40 kHz	1 / 5 / 20 ms	about 1.9 cm

The letter gives the frequency tier (detail against reach); S, M and L give the pulse length (energy per ping). L-L is the best-effort fallback: the lowest frequency with the longest pulse.

5.4 How a decision is made

1. Sound speed from Mackenzie.
2. For each profile: absorption at its centre frequency (Francois-Garrison), turbidity loss and spreading loss give the transmission loss.
3. Noise: thermal plus wind plus the receiver's noise figure.
4. The power needed for an output SNR of at least 12 dB plus the 3 dB margin.
5. Check the limits: the power must be within the transmitter's 20 W peak, the energy within what the battery allows for this ping, and the sweep within the band.
6. The energy each workable profile would spend: just enough power, never below 5 % amplitude.
7. Apply the detail-versus-energy rule (5.5) to pick the profile.
8. Choose the modulation from the target's motion (5.6).
9. Convert everything into the FPGA's own units and build the packet.

5.5 The detail-versus-energy rule

- Take the **finest profile whose energy is at most the larger of**: 4 times the energy of the cheapest profile that works, or 1 % of this ping's energy budget.
- In words: extra detail is worth up to four times the minimum energy, and energy below 1 % of the budget is too small to be worth saving.
- The budget shrinks with the battery (0.4 J per ping at full charge, falling linearly to a 10 % floor), so a low battery leans towards saving energy.
- Because a lower frequency always needs less energy, the rule in practice only ever picks X-S, H-S, M-S, L-S, L-M or L-L.
- If no profile closes the link, the transmitter sends L-L at the maximum allowed power and flags the ping as best effort.

5.6 Modulation from motion

- Doppler shift: $f_d = v / c \times f_c$, doubled in echo mode. The drift over one pulse is $f_d \times T$ cycles.
- **BPSK** while the drift is at most 0.25 cycle (cleanest when still); **LFM** up to 1 cycle; **geometric sweep** beyond that.
- Example: at 440 kHz with a 1 ms pulse, a target moving at 0.5 m/s drifts about 0.15 cycle, so BPSK is kept; at 2 m/s it drifts about 0.6 cycle, so LFM is used.

5.7 Stability: no flip-flopping

To move up to a finer profile than the one a contact is currently using, the link must close with 2 dB of extra margin. Moving down happens immediately. The memory is kept per contact, so a noisy sensor at a profile boundary cannot make the waveform jump back and forth from ping to ping.

5.8 Real-time structure

Task	Rate	Priority	Job
spiTask	50 Hz	4 (highest)	Sends the latest packet to the FPGA, so a new decision always arrives well before the next ping
decisionTask	6 Hz	3	Steps the simulation, runs the networks and the physics, publishes the packet. Never touches SPI or the console, so it can never be blocked by them
sensorTask	50 Hz	2	Reads and smooths the potentiometers or passes on typed inputs (idle in automatic mode)
consoleTask	On demand	1 (lowest)	Serial commands, the per-ping log and reports

Tasks share data through length-one queues that always hold the latest value, so a slow task can never build up a backlog. If sensor data is older than 500 ms, decisions pause and the last good packet keeps going.

5.9 Serial console

Command	What it does
A	Automatic: simulated conditions (the default at power-on), one log line per ping
I T S D turb bat R v	Type all seven inputs on one line and hold that condition; prints the full decision report
W	Asks for the inputs one by one
M lfm geo bpsk cw auto	Force a modulation, or return to choosing it from the motion
R <metres>	Set the target range
C [amp] and S [n] [r]	Record the DAC output in slow motion with the ESP32's ADC (section 9.4)

6. Machine Learning Model

6.1 Role: the network proposes, the physics verifies

- Three small neural networks run on the ESP32 for every ping and propose the profile, the amplitude and the modulation.
- The physics keeps the proposed profile only if it closes the link within the energy allowance; otherwise it chooses itself.
- It raises the proposed amplitude if it is too low for the link, and caps it at what the battery allows.
- It replaces a proposed modulation that cannot tolerate the target's Doppler drift.
- Every override is recorded in the packet's flags, so it is visible in the logs and in every capture.

6.2 Networks

Network	Shape	Inputs	Output
Profile	6 → 48 → 48 → 6 (ReLU)	Temperature, salinity, depth, turbidity, battery, log10(range), standardised	One score per profile the rule can pick
Amplitude	12 → 48 → 48 → 1	The same six inputs plus the chosen profile (one-hot)	log(amplitude that just closes the link), plus a 2.3 % safety margin
Modulation	16 → 32 → 32 → 3	Temperature, salinity, depth, log10(speed + 0.01) plus the profile (one-hot of 12)	LFM, geometric or BPSK

Together they hold 7,706 weights (30 KB), generated as a C header straight from training. The C code gives the same answers as the Python models on all 6,000 test vectors of each network.

6.3 Training data

- 300,000 scenarios labelled by a Python copy of the firmware's physics, half of them with the range drawn evenly on a log scale so short and long ranges are both well covered.
- Inputs rounded to 32-bit floats, as on the ESP32.
- The modulation network trains on 200,000 random combinations of conditions, speed and profile.
- Tested on two separate sets of 20,000 scenarios generated with different seeds.

6.4 Results

Model	Size	Same profile as the physics	Picks overridden
Decision tree (depth 14)	10,707 nodes	93.8 % / 96.0 %	4.9 %
XGBoost (depth 5, 60 rounds)	19,864 nodes	97.1 % / 98.1 %	2.2 %
XGBoost with product features	20,906 nodes	98.0 % / 98.7 %	1.6 %

Model	Size	Same profile as the physics	Picks overridden
Neural network (used)	2,982 weights	99.2 % / 99.4 %	0.5 %

Agreement is given for the evenly spread range set and the log-spread range set. Further results:

- Amplitude: within 1.8 % (median) of what the link needs, costing on average 1.6 to 3.4 % more energy than the minimum; the physics raises it in 1 to 2 % of cases.
- Modulation: agrees with the rule on 99.7 % of 20,000 test cases, with 0.2 % overridden. Every disagreement lies within about 1 % of a threshold.
- Per-class recall for the profile network is 98 to 100 % for every profile it can choose.

6.5 Why small neural networks

The decision boundaries are smooth curves in the input space. A small network fits them with a few thousand weights, while tree models need around 20,000 nodes and still agree less often. The network also fits comfortably in the ESP32's flash and runs in well under a millisecond.

6.6 Limits and the path to beating the physics

The labels come from the physics model, so today the network can only copy it. It becomes better than the physics only when it is retrained on measured results: logging the conditions, the choice and the received level of each ping, then training on what actually worked (section 15).

7. ESP32 to FPGA Link

7.1 SPI

- SPI mode 0, 2 MHz, most significant bit first. The ESP32 sends the latest packet as one 57-byte burst (228 μ s) every 20 ms.
- Packets are framed by the pause after each burst: a frame ends when the clock has been quiet for 100 μ s. On our boards the chip-select wire only ever carried noise, while the clock and data lines were clean, so we designed the receiver not to depend on it. Chip select can be switched back on with one parameter.
- Any byte count other than 57, a wrong sync byte or version, or a bad CRC means the packet is rejected and counted.

7.2 Packet format (version 3, 57 bytes, little-endian)

Bytes	Field	Meaning
0	sync	0xA5
1	version	3
2 to 3	seq	Increments with every new decision
4	profileId	0 to 11
5	modulation	Used by the FPGA: 0 tone, 1 LFM, 2 geometric, 3 BPSK
6	spreadingFactor	$\text{round}(\log_2(\text{bandwidth} \times \text{pulse length}))$ for sweeps
7	flags	Link OK, low battery, input clamped, ML profile used, best effort, ML profile overridden, ML amplitude raised, ML modulation overridden
8 to 27	telemetry	Centre frequency, bandwidth, pulse length, amplitude, power, propagation delay
28 to 31	lenClk	Pulse length in FPGA clocks
32 to 35	ftwStart	Start frequency as a tuning word
36 to 39	ftwStep	Sweep step (LFM: per clock $\times 2^{16}$; geometric: ratio per 32 clocks)
40 to 43	winStep	$2^{32} / \text{lenClk}$, so the window makes exactly one lap per pulse
44 to 45	ampQ12	Amplitude, $4096 = 1.0$
46 to 47	code	BPSK phase code (Barker-13 = 0x0A60)
48 to 51	chipLen	BPSK clocks per chip
52 to 55	periodClk	Clocks between pulse starts (1/6 s at 6 pings a second)
56	crc8	CRC-8, polynomial 0x07, over bytes 0 to 55

The ESP32 sends every waveform parameter **already in the FPGA's own units**, so the FPGA never has to divide. The 16 fractional bits of the LFM step matter: a 20 ms, 40 kHz chirp needs a step of 3.416 tuning-word units per clock, and rounding it to a whole number would sweep 12 % too little.

7.3 The PC as a second source

The PC can send the same packet over the board's USB UART (3 Mbaud), which lets us test any waveform without the ESP32. Over the same link the PC can ask the FPGA to capture one pulse, or every pulse (stream mode), and send it back for checking.

8. FPGA Waveform Engine

8.1 Clock and units

- System clock 50,142,857 Hz from the PLL.
- Frequency tuning word: $f \times 2^{32} / F_{\text{CLK}}$. One hertz is about 85.65 units.
- Phase is a 32-bit fraction of a cycle; the 1,024-entry sine table is indexed by its top 10 bits.
- Amplitudes and window values are fixed point, with 4096 meaning 1.0.

8.2 Signal chain, one sample per clock

1. A pulse timer fires a start every period. The pulse controller holds the engine active for exactly the pulse length.
2. The sweep unit produces the instantaneous frequency (constant, linear or geometric).
3. The phase accumulator adds it every clock. For BPSK, the phase code adds half a cycle (180°) on the chips marked in the code.
4. The sine table turns the phase into a 12-bit sample.
5. A second phase accumulator steps through the Hann window, exactly one lap per pulse.
6. Two multipliers apply the window and then the amplitude. Each takes two clocks: the factor is split into two parts whose products are registered, then added.
7. The result drives the sigma-delta DAC, the parallel DAC bus and the capture unit.

8.3 Sweeps

- **LFM:** a 48-bit register holds the tuning word with 16 fraction bits; every clock it adds the signed step. Long, narrow chirps therefore sweep exactly the bandwidth asked for.
- **Geometric:** every 32 clocks the register is multiplied by $(1 + \text{step} / 2^{32})$. The multiply is done by shift-and-add, one bit per clock, so no wide multiplier is needed.

8.4 BPSK

A chip counter advances every chip length; the phase flips by 180° whenever the current chip's bit in the code is 1. Barker-13 (+ + + + - - + - - +) is stored as 0x0A60, and the pulse is always exactly 13 whole chips.

8.5 Settings path

- A packet from the ESP32 or the PC is checked one byte per clock: sync, version and CRC.
- A good packet goes into a pending register. It is copied into the active settings only while no pulse is playing, so a pulse is never changed halfway through.
- The packet's sequence number, profile and flags travel with its settings, and a 16-bit counter numbers every pulse, so every capture states exactly which decision and which pulse it holds.

8.6 Capture and streaming

- The capture unit records 16,384 samples of the next pulse into block RAM, keeping one sample in 2^n where n is the smallest value that fits the whole pulse (one in 4 for 1 ms pulses, one in 64 for 20 ms).
- It then sends a 43-byte header (all the pulse's settings, packet and pulse numbers, good and bad packet counts) and the samples over the UART at 3 Mbaud, in about 0.11 s.
- In stream mode it re-arms after every capture, so all 6 pings of every second reach the PC.

8.7 Limits

Quantity	Limit
Pulse length	24 bits: up to about 335 ms
Period between pulses	32 bits of clocks: up to about 85 s
LFM sweep rate	Up to about 19 GHz/s (the sonar profiles need at most 0.12 GHz/s)
Phase code	Up to 16 chips (Barker-13 used)
Frequency	Numerically up to about 20 MHz; the sonar band is 100 to 500 kHz

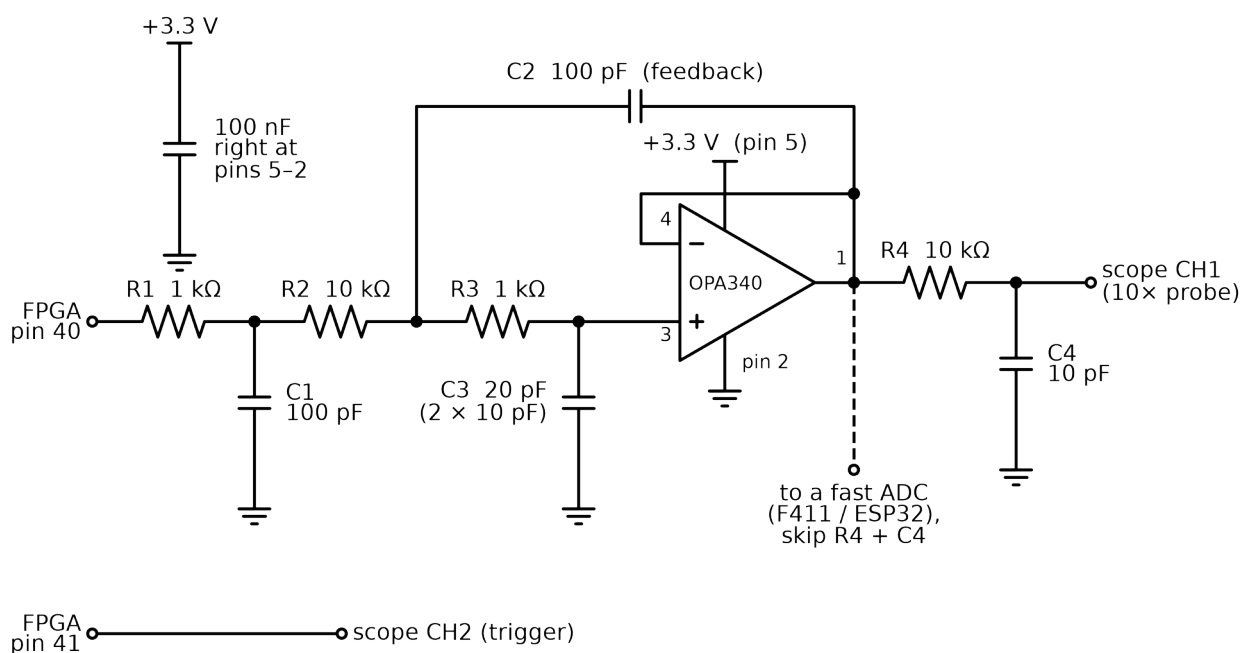
9. DAC and Analog Output

9.1 Second-order sigma-delta modulator

- Two integrators run every clock: $i1 += x - fb$, then $i2 += i1 - fb$, where x is the sample and fb is ± 2048 for the previous output bit. The output bit is 1 when $i2$ is zero or positive.
- The noise transfer is $(1 - z^{-1})^2$, so the quantisation noise is pushed far above 100 to 500 kHz, where the filter removes it.
- The input is limited to $\pm 95\%$ of full scale. In simulation this keeps the integrators bounded (below 10,000 and 50,000), whereas an input held at full scale made the second integrator grow without limit.
- The original first-order version is still available through a parameter, so the two can be compared on the same hardware.
- Both versions are checked bit for bit against Python models of the modulator in simulation.

9.2 Filter

- **Stage 1, RC:** 1 k Ω and 100 pF (corner about 1.6 MHz) removes the fastest switching before the op-amp sees it.
- **Stage 2, Sallen-Key low-pass:** 10 k Ω and 1 k Ω in series into the OPA340's + input, 20 pF from that input to ground, and 100 pF from the junction of the two resistors to the op-amp output. The op-amp is wired as a unity-gain follower. Cut-off about 1.1 MHz, Q about 0.64, nearly flat across the band.
- **Stage 3, RC:** 10 k Ω and 10 pF after the op-amp, for the oscilloscope and the ESP32's ADC.
- The op-amp output also drives the Black Pill's ADC directly, acting as the buffer a fast ADC needs.



All grounds: FPGA GND. Stage 1: 1 k Ω + 100 pF (1.6 MHz). Stage 2: Sallen-Key, ~ 1.1 MHz, $Q \approx 0.64$. Stage 3: 10 k Ω + 10 pF.

Figure 2. The analog output stage: RC stage, OPA340 Sallen-Key low-pass filter and output RC stage.

9.3 Measured at full speed

We recorded the same three fixed pings with the Black Pill, first with the first-order modulator and then with the second-order one. The difference is the measured output minus the FPGA's samples, after removing each recording's DC offset and fitting its gain.

Ping	Match, first order	Match, second order	Difference, first order	Difference, second order
L-S BPSK, 120 kHz, amp 0.56	0.997	0.998	28.0 mV rms	21.8 mV rms
H-S geometric, 270 to 330 kHz, amp 0.22	0.995	0.996	15.6 mV rms	12.8 mV rms
X-S BPSK, 440 kHz, amp 0.05	0.868	0.926	17.2 mV rms	12.5 mV rms

- "Match" is the correlation between the measured trace and the FPGA's own samples (1 is perfect).
- The second-order modulator lowers the remaining error by 20 to 30 % and lifts the hardest case, a faint 440 kHz ping, from 0.87 to 0.93.
- About 9 to 10 mV rms of what remains is the Black Pill's own noise floor, measured on the quiet baseline between pulses.
- The output height is 1.00 of the intended value at 120 kHz, 0.95 at 300 kHz and 0.86 at 440 kHz. Lengthening the ADC's sampling time five-fold did not change these figures, so the loss is real and in the analog path. Adding about 10 pF of stray wiring capacitance at the op-amp's input to the filter model reproduces it almost exactly (0.99, 0.94 and 0.84).

9.4 Slow-motion recording with the ESP32

Before the Black Pill, we verified the DAC with the ESP32's own ADC, which manages only about 80,000 samples a second. The ESP32 sends the FPGA a copy of the current ping with every frequency divided by up to 100 and the pulse made that much longer, records it, and the PC lines the recording up with the FPGA's samples. At full amplitude this gave a correlation of 0.998, with the BPSK phase flips clearly visible. It shows the waveform's shape, but not the filter's behaviour at the real frequencies, which is why the full-speed recording above was added.

10. Verification Strategy

10.1 The golden model is the specification

A Python model reproduces the FPGA's datapath bit for bit, including every pipeline delay and every rounding step. Every change to the hardware is mirrored in the model, every simulation is compared against it, and every capture from the real board must match it on every sample.

10.2 Simulation testbenches

Testbench	What it checks
phase_acc, pulse_ctrl, phase_code	Module behaviour, self-checking
capture	Header and samples at three decimation settings, and two streamed captures after one arm
dds	Tone against the golden model, plus the spectrum's peak
pulse_dds	Windowed pulses against the golden model; the window narrows the spectrum
chirp	LFM up and down, bit-exact, and the measured sweep
geo	Geometric sweep, bit-exact, and exactly two octaves measured
bpsk	Barker-13, bit-exact, and the code read back from the samples
sigma_delta	First- and second-order bit streams bit-exact against their models; the filtered output follows the waveform
top	The whole board: a chirp over the UART, an SPI packet with a broken CRC (rejected) and a good geometric packet, BPSK over the UART, and three captures checked end to end

All 11 testbenches pass.

10.3 Firmware tests

- 43 automatic checks on the PC: the Mackenzie check value, Francois-Garrison against an independent implementation, CRC-8 against its standard check value, packet size and band limits, the Doppler rule, the anti flip-flop margin, the detail-versus-energy rule, and the simulation's repeatability.
- Behaviour sweeps: a longer range, a lower battery or more turbidity never produces a finer profile; invalid inputs are handled.
- The C++ firmware and its Python copy agree on 5,000 random scenarios: no profile or modulation mismatches, amplitudes within 4e-6 and SNR within 7e-5 dB.
- The C neural networks match the Python models on all test vectors.

10.4 On-board tools

Tool	What it shows
demo.py	Streams 6 consecutive pings from the FPGA, checks each against the golden model and plots one second of the running system with the conditions behind each ping
wavegen.py, capture.py	Set any waveform from the PC, capture it back, check it and plot its waveform and spectrogram
real_speed.py	Records 6 consecutive pings after the DAC at 2.4 MS/s with the Black Pill and plots them over the FPGA's samples
dac_view.py, dac_sequence.py	The slow-motion DAC recordings from the ESP32's ADC
energy_compare.py	The adaptive design's energy against fixed sonars over a full battery cycle

11. Results

11.1 The whole system, bit-exact

With the ESP32 running the simulated mission, six consecutive pings are streamed back from the FPGA. Each is a new decision for a different contact, and every one matches the golden model on every sample.

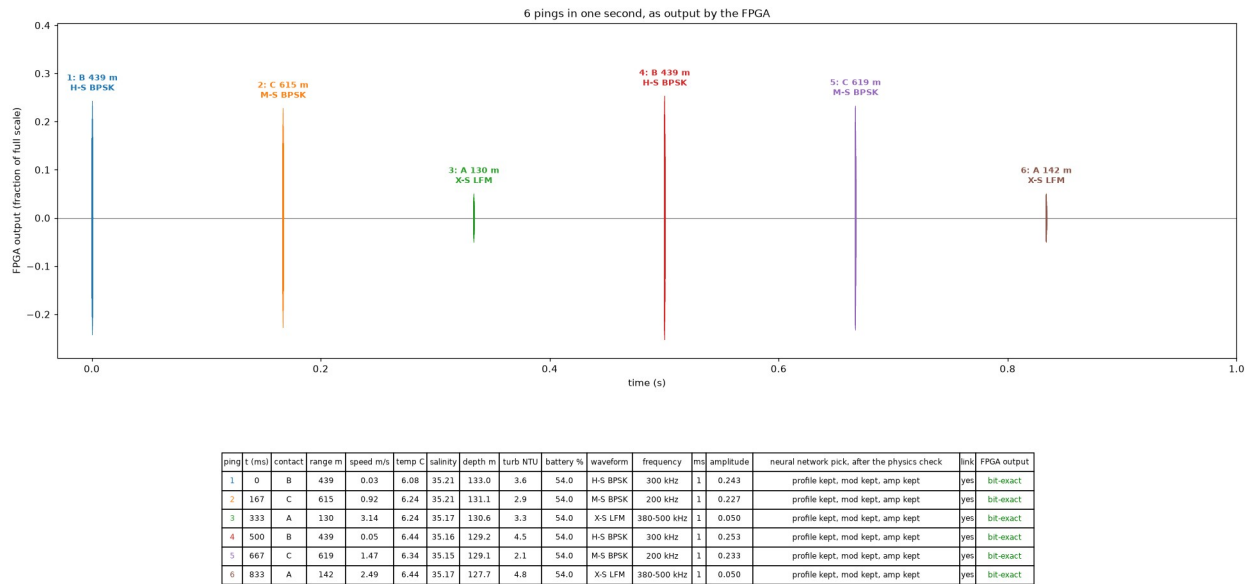


Figure 3. One second of the running system: six pings chosen by the ESP32 for three contacts, as output by the FPGA. Every ping is bit-exact against the golden model.

11.2 Waveforms captured on the board

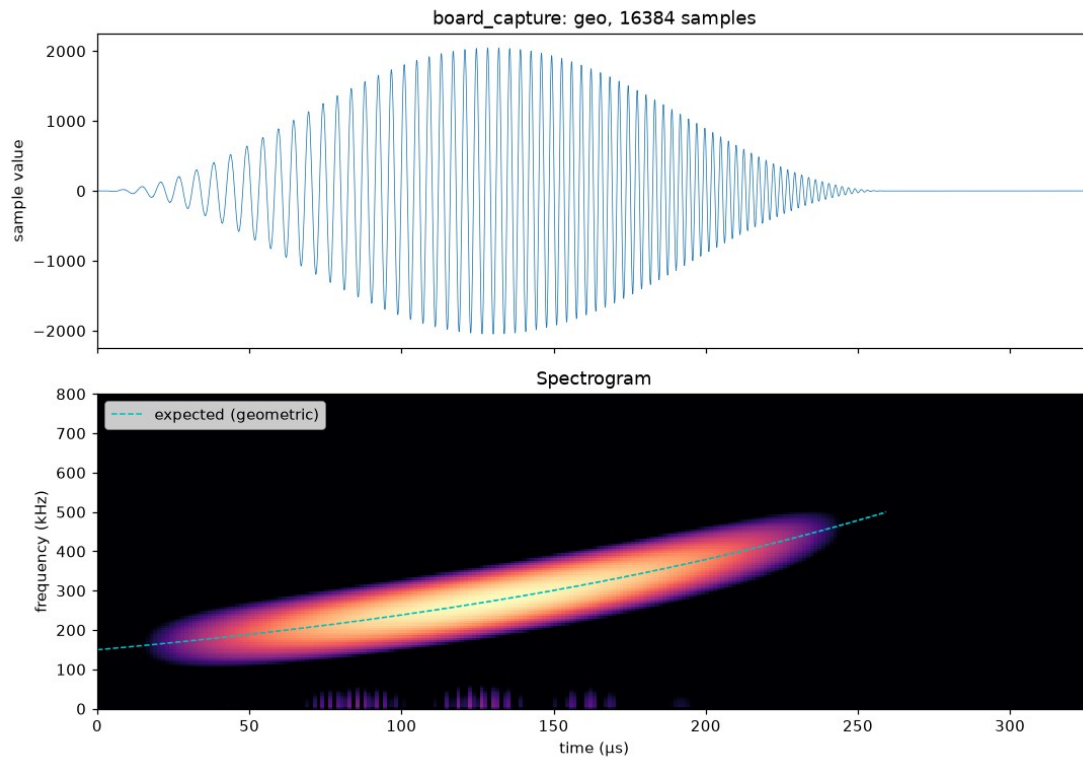


Figure 4. A geometric sweep from 150 to 500 kHz captured from the FPGA: the Hann-windowed waveform (top) and its spectrogram, with the expected sweep as a dashed line (bottom).

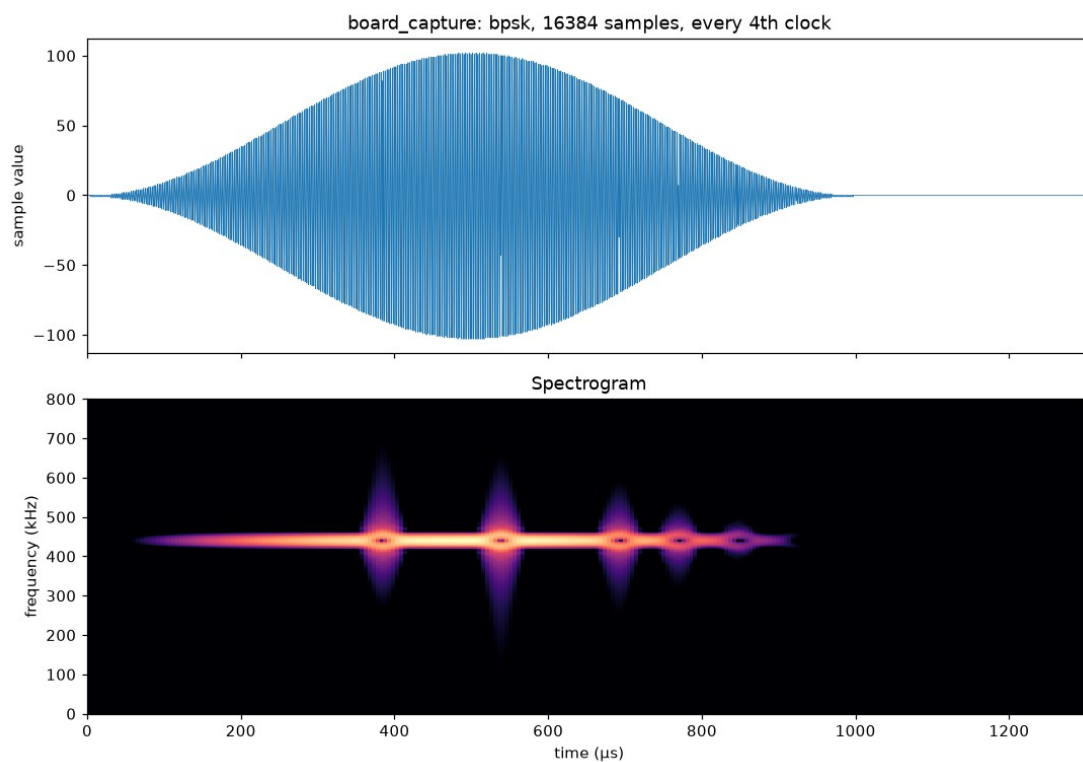


Figure 5. A Barker-13 BPSK pulse at 300 kHz captured from the FPGA.

11.3 Energy

Over one full battery cycle of the simulated mission (10,200 pings, about 28 minutes at 6 pings a second):

Transmitter	Average energy per ping	Link closes on
Adaptive (this project)	1.29 mJ	100 % of pings
Fixed L-L (120 kHz, 20 ms) at full allowed power	230 mJ (178 × more)	100 % of pings
Fixed H-S (300 kHz, 1 ms) at full power	20 mJ (15 × more)	70.7 % of pings

- The adaptive transmitter chose X-S on 33.3 % of pings, M-S on 24.0 %, L-S on 22.5 %, H-S on 18.9 %, L-M on 1.2 % and L-L on 0.2 %.
- A fixed sonar must either be sized for the worst case and waste energy on every easy ping, or be economical and lose the target when conditions get hard. The adaptive design does neither.
- These figures use the physics rule without the neural networks, which agree with it on more than 99 % of choices.

11.4 The analog output at full speed

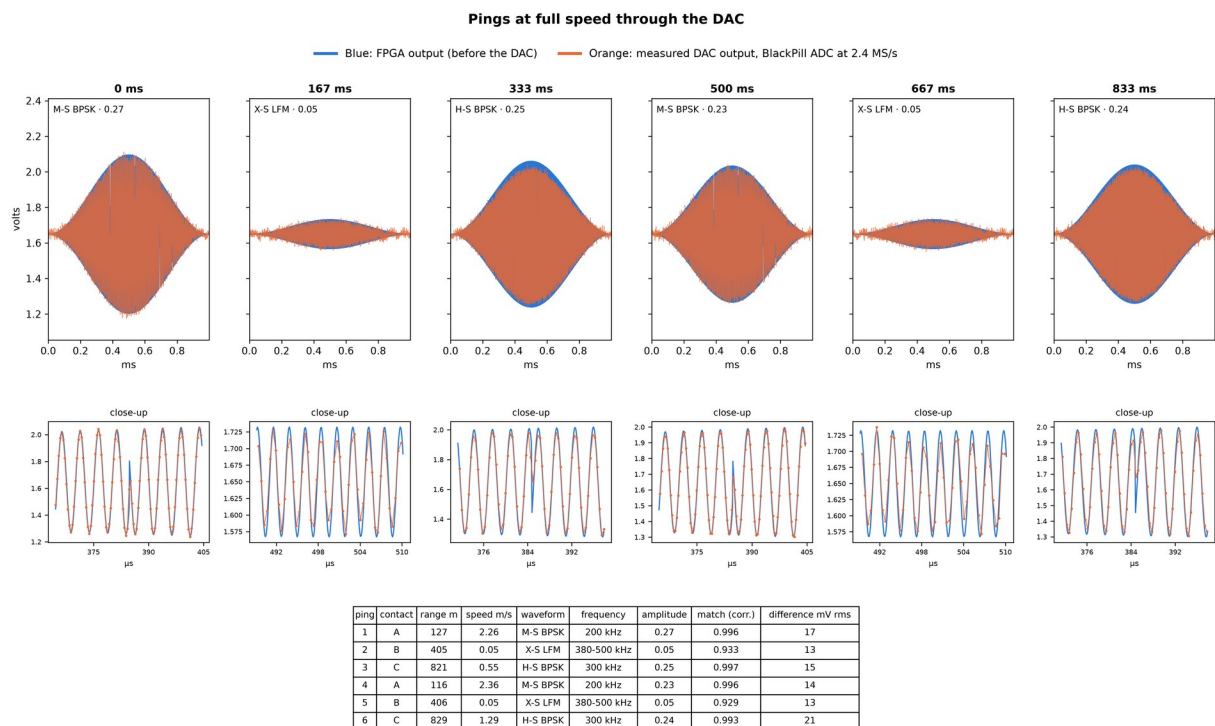


Figure 6. Six consecutive pings after the second-order sigma-delta and the Sallen-Key filter, recorded at 2.4 MS/s by the Black Pill (orange), over the FPGA's own samples (blue). Close-ups show individual carrier cycles and the BPSK phase flips.

12. Bill of Materials and Cost

Prices are indicative ranges from Indian hobby electronics suppliers and vary with supplier and quantity. Confirm before ordering.

Subsystem	Component	Approx. cost (India)	Notes
Decision controller	ESP32-S3 development board	About Rs. 700 to 1,200	Any ESP32-S3 board with native USB
Waveform engine	Sipeed Tang Nano 9K	About Rs. 1,800 to 2,500	Gowin GW1NR-9 FPGA, on-board USB JTAG and UART
Analog output	OPA340 op-amp	About Rs. 150 to 300	Single supply, rail-to-rail
Analog output	Resistors, capacitors, perfboard, wires	Under Rs. 150	1 k Ω , 10 k Ω , 10 pF, 100 pF, 100 nF
Test equipment	Black Pill STM32F411CE	About Rs. 450 to 700	Used for measurement, not part of the payload
Prototype total		About Rs. 3,000 to 4,800	Excluding test equipment, power amplifier and transducer

For the final payload, the parallel DAC, power amplifier and transducer still have to be selected (section 15). The FPGA and the decision firmware do not change when they are added.

13. Bench Testing Performed

13.1 Timeline

Date	Test	Result
29 Sep 2026	FPGA on the board, waveforms set from the PC	LFM, geometric, BPSK and a half-amplitude tone all matched the golden model on all 16,383 captured samples
30 Sep 2026	ESP32 to FPGA over SPI	10 packets a second, none rejected; the FPGA played every decision bit-exact
30 Sep 2026	Timing fix on the chip	A single-clock multiply gave occasional wrong samples on the real chip although timing analysis passed; splitting it into two registered steps made 5 of 5 captures bit-exact
30 Sep 2026	Whole system demo	Simulated conditions, 6 pings a second, 6 consecutive pings streamed and all bit-exact; the PC's copy of the simulation matched the ESP32's log
4 Oct 2026	DAC in slow motion (ESP32 ADC)	Correlation 0.998 at full amplitude; BPSK phase flips visible
4 Oct 2026	Six pings through the DAC	Matches of 0.92 to 0.999 after averaging 4 recordings each
5 Oct 2026	Sallen-Key filter and full-speed recording	Real-speed pulses on the laptop at 2.4 MS/s, matches up to 0.995
5 Oct 2026	Second-order sigma-delta	20 to 30 % less error on the same fixed pings; the faint 440 kHz ping improved from 0.87 to 0.93
5 Oct 2026	Source of the high-frequency loss	Not the ADC (unchanged with 5 times longer sampling); matches about 10 pF of stray capacitance

13.2 Test status

Test	Status
Waveform engine bit-exact on the board	Complete
ESP32 decisions sent and played without errors	Complete
Whole system, 6 pings a second, streamed and checked	Complete
DAC output recorded at full speed	Complete
Second-order sigma-delta compared with first order	Complete
Oscilloscope confirmation of the filter output	Outstanding at the time of writing
Real water sensors in place of the simulation	Outstanding
Power amplifier and transducer	Outstanding
In-water test of the link	Outstanding

14. Limitations and Honest Ledger

Each part of the system is listed with the strongest evidence we actually have for it, to avoid claiming more than we have shown.

Part	Evidence	What it does not show yet
Waveform engine (FPGA)	Verified on hardware: every sample matches the golden model	Nothing outstanding for the digital output
ESP32 decisions and link	Verified on hardware: 6 a second, no bad packets, every decision played bit-exact	Behaviour with real sensors
Physics model	Published models, cross-checked against independent implementations and tested in software	Validation in real water
Turbidity loss	Anchored to one published measurement	Needs calibration with a turbidity sensor and a known-range test
Doppler thresholds	Engineering rules of thumb (0.25 and 1 cycle)	Need confirming against measured echoes
Neural networks	Agree with the physics on more than 99 % of tested scenarios	Cannot outperform the physics until trained on measured data
Analog output	Measured at full speed with a 2.4 MS/s ADC	Independent oscilloscope check; the final parallel DAC
Water conditions	Deterministic, realistic simulation	Real sensors
Power stage	Not built	Everything

14.1 Known limitations

- The model treats the link as one-way (the same convention as the training data). Echo mode, with twice the loss and a target strength, is implemented but not yet the default; the team must decide which applies to the mission.
- One transducer rarely covers a 5:1 frequency span such as 100 to 500 kHz. The firmware assumes it can; separate transducers per band would need the profile table and the FPGA's output switching to reflect that.
- Absorption is evaluated at each profile's centre frequency rather than averaged across the band; the 3 dB margin covers most of the difference.
- The analog output loses about 14 % of its height at 440 kHz because of stray capacitance in the perfboard wiring. A proper PCB, or a smaller 10 pF capacitor in place of 20 pF, would flatten the response.
- The ESP32 and the FPGA each keep their own 6 Hz ping clock, so about once every few hours a ping may repeat or skip a decision. A trigger line from the FPGA would synchronise them.
- The chip-select wire between the boards carries only noise. The link works without it, but the cause was not found.
- The target's range and speed are simulated. In service they must come from the sonar's own echoes or another sensor such as a DVL.

14.2 Scope

This prototype covers the transmitter's intelligence and waveform generation, from water conditions to an analog signal, verified on the bench. The power amplifier, transducer, receiver and in-water testing are outside what has been built so far and are the next stages of the roadmap.

15. Roadmap

1. **Oscilloscope confirmation** of the filter output against the Black Pill's measurements.
2. **Parallel DAC:** a single-supply high-speed DAC (DAC908 or AD9708 class) on the existing 8-bit bus and sample clock, replacing the 1-bit stage for transmission.
3. **Power amplifier and transducer:** a wideband transducer, or one per band, with output switching in the FPGA.
4. **Real sensors** for temperature, salinity, depth and turbidity in place of the simulation.
5. **Calibration in water:** turbidity loss, absorption and the Doppler thresholds against a known-range target.
6. **Closing the loop for the neural networks:** log the received level of each ping with its conditions and choice, and retrain on what actually worked. This is the step at which the networks can do better than the physics model.
7. **Range and speed from echoes:** estimate them from the sonar's own returns instead of a mission setting.
8. **Synchronisation:** a trigger line from the FPGA so that decisions and pings share one clock.
9. **Tuning with the team:** the detail-versus-energy settings (4 times the minimum energy, 1 % of the budget) and one-way versus echo operation, followed by retraining.

16. How to Build and Run

Everything runs from the root of the repository. The FPGA flow is fully open source.

Step	Command
Run all FPGA simulations	sim/all
Build and load the FPGA (volatile)	tn9k load
Build and write the FPGA's flash	tn9k flash
Build the ESP32 firmware	arduino-cli compile with board esp32s3 (CDCOnBoot=cdc, 16 MB flash); see AGENTS.md
Flash the Black Pill (in DFU mode)	pio run -t upload in blackpill/
One second of the running system	python3 host/demo.py
Pings at full speed after the DAC	python3 host/real_speed.py
Energy comparison	python3 host/energy_compare.py
Set and capture a waveform from the PC	python3 host/wavegen.py geo --fc 325e3 --bw 350e3 --capture
Firmware tests on the PC	bash tools/run_tests.sh (inside the firmware package)
Retrain the neural networks	python ml/train.py (about 5 minutes on a laptop CPU)

The ESP32 starts in automatic mode at power-on and needs no PC. Only the FPGA has to be connected for the demo. The full details, including toolchain versions and hardware quirks, are in AGENTS.md in the repository.

17. Repository Layout

Path	Contents
top.v and the other .v files	The FPGA design: packet input and checking, waveform engine, capture, sigma-delta DAC
sim/	Testbenches and the scripts that run them
host/	Golden model, packet tools, demo, capture, DAC recording and energy scripts
MCU /SIH/	ESP32-S3 firmware: physics, decision rules, simulation, neural networks, tests and the firmware write-up
MCU /SIH/ml/	Training code, report and test vectors for the neural networks
blackpill/	Firmware for the 2.4 MS/s recorder
AGENTS.md	The full engineering guide: design, packet, pins, toolchain quirks, status and open items

18. References

1. Mackenzie, K. V. (1981). Nine-term equation for sound speed in the oceans. *Journal of the Acoustical Society of America*, 70(3), 807-812.
2. Francois, R. E., and Garrison, G. R. (1982). Sound absorption based on ocean measurements. Part I: Pure water and magnesium sulfate contributions. *JASA*, 72(3), 896-907.
3. Francois, R. E., and Garrison, G. R. (1982). Sound absorption based on ocean measurements. Part II: Boric acid contribution and equation for total absorption. *JASA*, 72(6), 1879-1890.
4. Mellen, R. H. (1952). The thermal-noise limit in the detection of underwater acoustic signals. *JASA*, 24(5), 478-480.
5. Wenz, G. M. (1962). Acoustic ambient noise in the ocean: spectra and sources. *JASA*, 34(12), 1936-1956.
6. Richards, S. D., Heathershaw, A. D., and Thorne, P. D. (1996). The effect of suspended particulate matter on sound attenuation in seawater. *JASA*, 100(3), 1447-1450.
7. Urick, R. J. (1983). *Principles of Underwater Sound*, 3rd edition. McGraw-Hill.
8. Barker, R. H. (1953). Group synchronizing of binary digital systems. In *Communication Theory*, W. Jackson (ed.), Butterworths, 273-287.
9. Harris, F. J. (1978). On the use of windows for harmonic analysis with the discrete Fourier transform. *Proceedings of the IEEE*, 66(1), 51-83.
10. Tierney, J., Rader, C. M., and Gold, B. (1971). A digital frequency synthesizer. *IEEE Transactions on Audio and Electroacoustics*, 19(1), 48-57.
11. Schreier, R., and Temes, G. C. (2005). *Understanding Delta-Sigma Data Converters*. Wiley-IEEE Press.
12. Sallen, R. P., and Key, E. L. (1955). A practical method of designing RC active filters. *IRE Transactions on Circuit Theory*, 2(1), 74-85.
13. MB-System, mbabsorption: an independent implementation of the Francois-Garrison absorption model, used as a cross-check.
14. Texas Instruments. OPA340 single-supply, rail-to-rail operational amplifier, datasheet.